Sprint 1 Structured Notes: Web Language Roles and First JavaScript Tools (සිංහල පරිවර්තනය)

0. What Sprint 1 Is About

Concept Explanation (Sinhala): Sprint 1 කියන්නේ web development සඳහා JavaScript ඉගෙන ගන්න පටන් ගන්න පළමු පියවරයි.

මේ Sprint එකේ ප්‍රධාන අරමුණු තමයි:

1. Web page එකක් ඇතුළේ JavaScript වලින් සිද්ධ වෙන කාර්යභාරය තේරුම් ගැනීම.
2. HTML සහ CSS සමඟ JavaScript එකතු වෙලා වැඩ කරන්නේ කොහොමද කියා ඉගෙන ගැනීම.
3. ඔයාගේ පළමු JavaScript tools පාවිච්චි කරන ආකාරය ඉගෙන ගැනීම:
   - `console.log()`
   - comments
   - semicolons (`;`)
   - button clicks
   - `getElementById()`
   - `addEventListener()`
   - `alert()`
   - `textContent`
4. කුඩා interactive (ප්‍රතිචාර දක්වන) web page එකක් සාදා ගැනීම.
5. JavaScript වලට HTML element එකක් සොයාගත නොහැකි වූ විට සිදුවන පොදු වැරැද්දක් (beginner error) හඳුනාගෙන එය නිවැරදි කිරීම.

මෙම Sprint 1 අවසානයේදී, ඔයාට button එකක් තියෙන සරල HTML page එකක් හදන්නත්, ඒ button එක click කරනකොට JavaScript වලින් ප්‍රතිචාරයක් දක්වන්න සලස්වන්නත් පුළුවන් වෙනවා.

1. The Three Main Web Languages

Concept Explanation (Sinhala): සාමාන්‍යයෙන් basic web page එකක් හදන්න භාෂා 3ක් භාවිත වෙනවා. ඒ තමයි:

1. HTML
2. CSS
3. JavaScript

මේ තුනම එකතු වෙලා වැඩ කළත්, ඒ හැම එකකින්ම කරන්නේ එකිනෙකට වෙනස් වැඩයි.

1.1 HTML: Structure

Concept Explanation (Sinhala): HTML මඟින් web page එකට structure එකක් (සැකිල්ලක්) ලබා දෙනවා.

HTML වලින් මේ වගේ දේවල් නිර්මාණය කරනවා:

- headings (මාතෘකා)
- paragraphs (ඡේද)
- buttons (බොත්තම්)
- images (රූපරාමු)
- links (සබැඳි)
- forms (ෆෝරම)
- lists (ලැයිස්තු)

Code Block:

```
<h1>Hello</h1>
<p>This is a paragraph.</p>
<button>Click me</button>
```

Code Breakdown (Sinhala):

- `<h1>Hello</h1>` : මේකෙන් පිටුවේ ලොකු main heading එකක් හදනවා.
- `<p>This is a paragraph.</p>` : මේකෙන් සාමාන්‍ය text ලියන්න paragraph එකක් හදනවා.
- `<button>Click me</button>` : මේකෙන් click කරන්න පුළුවන් button එකක් හදනවා.

කෙටියෙන් කිව්වොත්, HTML උත්තර දෙන්නේ **"Web page එකේ තියෙන්නේ මොනවාද?"** කියන ප්‍රශ්නයටයි. ඒ නිසා HTML කියන්නේ හරියට website එකක skeleton (සැකිල්ල) එක වගේ දෙයක්.

1.2 CSS: Style

Concept Explanation (Sinhala): CSS මඟින් web page එක පෙනෙන විදිහ (style එක) පාලනය කරනවා.

CSS වලින් මේවා වෙනස් කරන්න පුළුවන්:

- colors (පාටවල්)
- font sizes (අකුරුවල ප්‍රමාණයන්)
- spacing (හිස්තැන්/පරතරයන්)
- borders (සීමා මායිම්)
- layout (පිටුවේ හැඩතල සැකැස්ම)
- background colors (පසුබිම් වර්ණ)

Code Block:

```
h1 {
  color: green;
}
```

Code Breakdown (Sinhala):

- `h1 { ... }` : HTML වල තියෙන `<h1>` heading එක select කරගන්නවා.
- `color: green;` : එම heading එකේ අකුරුවල වර්ණය කොළ පාට (green) කරනවා.

CSS උත්තර දෙන්නේ **"Web page එක පෙනෙන්න ඕනේ කොහොමද?"** කියන ප්‍රශ්නයටයි. ඒ නිසා CSS කියන්නේ හරියට website එකක ඇඳුම් පැළඳුම් හෝ design එක වගේ දෙයක්.

1.3 JavaScript: Behavior

Concept Explanation (Sinhala): JavaScript මඟින් web page එකට විවිධ ක්‍රියාකාරකම් කරන්න පුළුවන් හැකියාව (behavior) ලබා දෙනවා.

JavaScript වලින් මේ වගේ දේවල් පාලනය කරන්න පුළුවන්:

- button clicks (බොත්තම් click කිරීම්)
- alert messages (පොප්-අප් පණිවිඩ)
- menus open/close කිරීම්
- form හැසිරීම් (form validation)
- පිටුවේ තියෙන text වෙනස් කිරීම
- dynamic විදිහට page එක update කිරීම

JavaScript උත්තර දෙන්නේ **"Web page එකේ මොනවාද සිද්ධ වෙන්න ඕනේ?"** කියන ප්‍රශ්නයටයි. ඒ නිසා JavaScript කියන්නේ website එකක ක්‍රියාකාරීත්වය (action හෝ behavior) වගේ දෙයක්.

1.4 Simple Comparison

Concept Explanation (Sinhala): මේ භාෂා තුනේ සරල සංසන්දනයක් පහත වගුවෙන් බලමු:

Language	Job	Simple Meaning
HTML	Structure	පිටුවේ තියෙන්නේ කුමක්ද (What exists on the page)
CSS	Style	එය දිස්වන්නේ කෙසේද (How it looks)
JavaScript	Behavior	එහි සිදුවන්නේ කුමක්ද (What happens)

Code Block:

```
<h1>Hello, World!</h1>
<button onclick="alert('Button clicked!')">Click me</button>
```

Code Breakdown (Sinhala):

- `<h1>Hello, World!</h1>` : HTML මඟින් heading එකක් සාදයි.
- `<button>` : HTML මඟින් button එකක් සාදයි.
- `onclick="alert('Button clicked!')"` : JavaScript මඟින් button එක click කළ විට alert message එකක් පෙන්වීමට ක්‍රියාකාරීත්වය එකතු කරයි.

2. What JavaScript Adds to a Web Page

Concept Explanation (Sinhala): HTML සහ CSS වලින් විතරක් හදන page එකක් සාමාන්‍යයෙන් static (අකර්මණ්‍ය/නිශ්චල) පිටුවක් වෙනවා. ඒ කියන්නේ ඒක පරිශීලකයාගේ ක්‍රියාවලට ප්‍රතිචාර දක්වන්නේ නැහැ. JavaScript මඟින් web page එකක් interactive (පරිශීලකයා සමඟ ගනුදෙනු කළ හැකි) තත්ත්වයට පත් කරනවා.

උදාහරණයක් විදිහට:

1. User කෙනෙක් button එකක් click කරනවා.
2. JavaScript ඒ click එක හඳුනාගන්නවා (notices).
3. JavaScript ඊට අදාළ code එකක් run කරනවා.
4. Web page එක ඒකට ප්‍රතිචාර දක්වනවා.

මෙසේ දැක්විය හැකි ප්‍රතිචාර (responses) කිහිපයක්:

- alert එකක් පෙන්වීම.
- browser console එකේ පණිවිඩයක් print කිරීම.
- page එකේ තියෙන text වෙනස් කිරීම.
- menu එකක් open කිරීම හෝ close කිරීම.
- යම් element එකක් සැඟවීම (hide) හෝ දර්ශනය කිරීම (show).

Sprint 1 වල වැදගත්ම අදහස තමයි: **JavaScript මඟින් web page එකකට පරිශීලකයා (user) සමඟ ප්‍රතිචාර දැක්වීමට ඉඩ සලසයි.**

3. Your First HTML Page Structure

Concept Explanation (Sinhala): සරල HTML page එකක ව්‍යුහය (structure) මේ වගේ පෙනෙනවා:

Code Block:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>JavaScript Practice</h1>
    <p>Click the button to run JavaScript.</p>
    <button>Run</button>
  </body>
</html>
```

3.1 `<!DOCTYPE html>`

Concept Explanation (Sinhala): මේකෙන් browser එකට කියන්නේ: **"මේක HTML ලේඛනයක් (HTML5 document එකක්)"** කියලයි. මේ line එක සාමාන්‍යයෙන් HTML file එකක ඉහළින්ම ලියනවා.

3.2 `<html>`

Concept Explanation (Sinhala): `<html>` සහ `</html>` කියන්නේ මුළු HTML page එකටම තියෙන ප්‍රධාන container එකයි. පිටුවේ තියෙන හැමදේම ලියවෙන්නේ මේ tags දෙක ඇතුළෙයි.

3.3 `<body>`

Concept Explanation (Sinhala): `<body>` සහ `</body>` ඇතුළේ තියෙන්නේ web page එකේ ඇසට පෙනෙන සියලුම දේවල් (visible content). උදාහරණයක් විදිහට headings, paragraphs, buttons සහ images.

3.4 `<h1>`

Concept Explanation (Sinhala): `<h1>` tags මඟින් ප්‍රධාන මාතෘකාවක් (main heading) සාදයි. සාමාන්‍යයෙන් පිටුවක තියෙන වැදගත්ම මාතෘකාවට `<h1>` පාවිච්චි කරයි.

3.5 `<p>`

Concept Explanation (Sinhala): `<p>` tags මඟින් සාමාන්‍ය ඡේදයක් (paragraph text) සාදයි.

3.6 `<button>`

Concept Explanation (Sinhala): `<button>` tag එකෙන් click කළ හැකි බොත්තමක් හදනවා. නමුත් තනියම තිබෙන විට මෙයට ක්‍රියාකාරීත්වයක් නැත. එයට පණ දෙන්නේ JavaScript වලිනි.

4. The `<script>` Tag

Concept Explanation (Sinhala): අපිට JavaScript කේත ලියන්න පුළුවන් වෙන්නේ HTML පිටුව ඇතුළේ තියෙන `<script>` කියන tag එක ඇතුළෙයි.

Code Block:

```
<script>
  console.log("Hello from JavaScript");
</script>
```

Code Breakdown (Sinhala): Browser එක විසින් `<script>` tag එක ඇතුළේ තියෙන JavaScript code එක කියවා ක්‍රියාත්මක (run) කරනු ලබයි.

Full Example:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Hello</h1>
    <script>
      console.log("Page loaded");
    </script>
  </body>
</html>
```

පිටුව load වෙනකොටම, browser එක විසින් script එක ඇතුළේ තියෙන `console.log("Page loaded");` කියන කේතය ක්‍රියාත්මක කරයි.

5. Reading Code From Top to Bottom

Concept Explanation (Sinhala): JavaScript සාමාන්‍යයෙන් ක්‍රියාත්මක වෙන්නේ ඉහළ සිට පහළට (top to bottom), පිළිවෙළින් (in order).

Code Block:

```
console.log("Start");
console.log("Middle");
console.log("End");
```

Code Breakdown (Sinhala): මෙහි output එක පිළිවෙළින් මෙසේ දිස්වේ:

1. `Start`
2. `Middle`
3. `End`

වැඩසටහනක් run කිරීමට පෙර, එහි ප්‍රතිඵලය කුමක් විය හැකිදැයි අනාවැකි කීම (predict) හොඳ පුරුද්දකි. **Beginner Workflow එකක්:**

1. Code එක කියවන්න (Read).
2. සිදුවන්නේ කුමක්දැයි අනාවැකි පළ කරන්න (Predict).
3. Code එක run කරන්න (Run).
4. ඔයාගේ අනාවැකිය සහ ලැබුණු ප්‍රතිඵලය සසඳන්න (Compare).
5. වැරදි තිබේ නම් ඒවා නිවැරදි කරන්න (Fix).

6. `console.log()`

Concept Explanation (Sinhala): `console.log()` කියන්නේ මුලින්ම ඉගෙන ගන්න ඕනේ ප්‍රධානම JavaScript tool එකක්. මේකෙන් කරන්නේ කිසියම් පණිවිඩයක් හෝ අගයක් browser එකේ තියෙන **console** එකට print කරන එකයි.

Code Block:

```
console.log("Page loaded");
```

6.1 What Is the Console?

Concept Explanation (Sinhala): Console එක කියන්නේ browser එක ඇතුළේ තියෙන, සාමාන්‍ය පරිශීලකයින්ට පෙනෙන්නේ නැති, developers ලා සඳහාම වෙන්වූ tool එකක් (Developer Tool).

Developers ලා console එක පාවිච්චි කරන්නේ:

- Code එක පරීක්ෂා කිරීමට (test code)
- විවිධ අගයන් නිවැරදිදැයි බැලීමට (check values)
- පණිවිඩ බලාගැනීමට (see messages)
- වැරදි සොයාගැනීමට (find errors/debug)

6.2 `console.log()` Does Not Print on the Web Page

Concept Explanation (Sinhala): **ඉතා වැදගත්:** `console.log("Hello");` මඟින් "Hello" යන්න කෙලින්ම web page එක උඩ ප්‍රදර්ශනය කරන්නේ නැත. එය පෙන්වන්නේ browser console එක ඇතුළේ විතරයි.

Console එක open කරගන්නා ආකාරය:

- Web page එක මත Right click කර -> **Inspect** තෝරා -> **Console** tab එක click කරන්න.
- නැතහොත් keyboard shortcut එකක් භාවිත කරන්න: `Ctrl + Shift + J` (Windows) හෝ `Cmd + Option + J` (Mac).

6.3 Why `console.log()` Is Useful

Concept Explanation (Sinhala): අපේ code එක හරියට run වෙනවද කියලා පියවරෙන් පියවර (step by step) තහවුරු කරගන්න `console.log()` උදව් වෙනවා.

Code Block:

```
console.log("Page loaded");
console.log("Button ready");
console.log("Button clicked");
```

Code Breakdown (Sinhala):

- `Page loaded` ලෙස console එකේ වැටුණොත්: අපේ script එක සාර්ථකව සම්බන්ධ වී ඇත.
- `Button ready` ලෙස වැටුණොත්: JavaScript මඟින් අදාළ button එක සොයාගෙන ඇත.
- `Button clicked` ලෙස වැටුණොත්: button click එකේ ක්‍රියාව හරියටම සිදුවී ඇත.

7. Comments

Concept Explanation (Sinhala): Comments (අදහස් දැක්වීම්) කියන්නේ code එක ඇතුළේ ලියන සටහන් (notes) වලටයි. JavaScript විසින් මේවා ක්‍රියාත්මක කරන්නේ නැහැ (ignore කරනවා). Comments ලියන්නේ මිනිසුන්ගේ ප්‍රයෝජනය සඳහා පමණයි.

7.1 Single-Line Comments

Concept Explanation (Sinhala): තනි පේළියක comment එකක් ලියන්න `//` සංකේතය භාවිත කරයි.

Code Block:

```
// This is a comment
console.log("Hello");
```

Code Breakdown (Sinhala): මෙහිදී `// This is a comment` කියන කොටස browser එක මඟින් මඟහරින (ignore කරන) අතර, `console.log("Hello");` කොටස පමණක් ක්‍රියාත්මක කරයි.

7.2 Why Comments Are Useful

Concept Explanation (Sinhala): Comments අපේ code එක තේරුම් කරන්න උදව් වෙනවා. මේවා වැදගත් වෙන්නේ:

- යම් කේතයක් ලිව්වේ හේතුව පසුව මතක් කර ගැනීමට.
- වෙනත් developer කෙනෙකුට අපේ කේතය කියවා තේරුම් ගැනීමට.
- ඉගෙන ගන්නා අතරතුර විවිධ කොටස් ලේබල් (label) කර තැබීමට.

Code Block:

```
// Check that the script is connected correctly
console.log("Page loaded");
```

7.3 Good Comments vs Weak Comments

Concept Explanation (Sinhala): * **දුර්වල comment එකක් (Weak comment):**

```
// This logs hello
console.log("Hello");
```

(මෙය වැඩක් නැත, මන්ද කේතයෙන්ම එය පැහැදිලි වන බැවිනි)

- **හොඳ comment එකක් (Good comment):**

  ```
  // Test that JavaScript is running after the page loads
  console.log("Hello");
  ```

  (මෙය කේතය ලිවීමේ අරමුණ පැහැදිලි කරයි)

හොඳ comments වලින් හැමවිටම කියවෙන්නේ **"ඇයි මේ කේතය මෙතන තියෙන්නේ?" (Why)** කියන එක මිසක්, "මේ line එකෙන් මොකක්ද කරන්නේ" (What) කියන එක විතරක් නෙවෙයි.

7.4 Comments in HTML

Concept Explanation (Sinhala): HTML වල comment ලියන ක්‍රමය JavaScript වලට වඩා වෙනස් වේ.

Code Block:

```
<!-- HTML goal: show a heading, paragraph, and button -->
```

HTML comments ද browser එක මඟින් ක්‍රියාත්මක නොකරන අතර, ඒවා ලියන්නේ HTML කියවන අයගේ පහසුව සඳහාය.

8. Semicolons

Concept Explanation (Sinhala): JavaScript වල statement එකක් අවසාන වන තැන සලකුණු කිරීමට Semicolon (`;`) එකක් භාවිත කරයි.

Code Block:

```
console.log("Hello");
```

Semicolon එකෙන් JavaScript එකට කියන්නේ: **"මෙම උපදෙස (instruction) මෙතැනින් අවසන්"** කියලයි.

8.1 Statement

Concept Explanation (Sinhala): Statement එකක් කියන්නේ JavaScript වල තනි උපදෙසකටයි (single instruction).

Code Block:

```
console.log("Start");
console.log("Stop");
```

මෙහි statements 2ක් ඇත:

1. `Start` යන්න print කිරීම.
2. `Stop` යන්න print කිරීම. සෑම statement එකක්ම semicolon එකකින් අවසන් වී ඇත.

8.2 Do You Always Need Semicolons?

Concept Explanation (Sinhala): JavaScript වලට සමහර විට semicolons ස්වයංක්‍රීයව එකතු කරගත හැක (Automatic Semicolon Insertion). නමුත් ආරම්භකයකු ලෙස හැමවිටම statements අවසානයේ semicolon එකක් අනිවාර්ය දැමීම හොඳම සහ ආරක්ෂිතම පුරුද්දයි. එයින් කේතය කියවීමට පහසු වන අතර අනවශ්‍ය errors මඟහරවා ගත හැක.

9. The `id` Attribute

Concept Explanation (Sinhala): `id` එකක් මඟින් HTML element එකකට අනන්‍ය වූ (unique) නමක් ලබා දෙයි.

Code Block:

```
<button id="runBtn">Run</button>
```

මෙහි button එකට දී ඇති ID එක `runBtn` වේ. JavaScript වලට මෙම ID එක පාවිච්චි කරලා අදාළ button එක සොයාගත හැක. **රීතිය:** එකම ID එක එකම page එකක වෙනත් කිසිම element එකකට පාවිච්චි නොකළ යුතුය (IDs must be unique).

10. Finding an HTML Element with `getElementById()`

Concept Explanation (Sinhala): JavaScript වලට HTML element එකක් එහි ID එක ආධාරයෙන් සොයාගත හැක්කේ `document.getElementById("idName");` මඟිනි.

Code Block:

```
<!-- HTML -->
<button id="runBtn">Run</button>

<!-- JavaScript -->
<script>
  const button = document.getElementById("runBtn");
</script>
```

Code Breakdown (Sinhala):

- `document.getElementById("runBtn")` : HTML පිටුව පුරා සෝදිසි කර `runBtn` යන ID එක ඇති element එක සොයා ගනියි.
- `const button = ...` : සොයාගත් button element එක, පසුව භාවිතය සඳහා `button` නමැති variable එකක් (නමක්) තුළ තැන්පත් කර ගනියි.

10.1 What Is `document`?

Concept Explanation (Sinhala): `document` කියන object එකෙන් මුළු web page එකම නියෝජනය කරනවා. `document.getElementById(...)` ලියන විට අප JavaScript වලට පවසන්නේ: **"දැනට පවතින HTML document එක ඇතුළේ මේ ID එක තියෙන element එක සොයන්න"** කියලයි.

10.2 Why the ID Must Match Exactly

Concept Explanation (Sinhala): HTML වල ඇති ID එක සහ JavaScript එකේ සොයන ID එක අකුරෙන් අකුර සමාන විය යුතුය (case-sensitive).

- **නිවැරදි ක්‍රමය (වැඩ කරයි):**

  ```
  <button id="runBtn">Run</button>
  <script>
    const button = document.getElementById("runBtn");
  </script>
  ```

- **වැරදි ක්‍රමය (වැඩ කරන්නේ නැත):**

  ```
  <button id="runBtn">Run</button>
  <script>
    const button = document.getElementById("wrongId");
  </script>
  ```

  මෙහි `wrongId` නමින් element එකක් නැති නිසා JavaScript වලට එය සොයාගත නොහැක.

11. `const` in Sprint 1 Code

Concept Explanation (Sinhala): Sprint 1 වලදී variables ගැන ගැඹුරින් ඉගෙන නොගත්තත්, `const` භාවිත කරන සරල හේතුව මෙසේ වටහා ගන්න:

Code Block:

```
const button = document.getElementById("runBtn");
button.addEventListener(...);
```

Code Breakdown (Sinhala): මෙහිදී `const button` මඟින් සොයාගත් element එක `button` කියන නම යටතේ මතක තබා ගනියි. එවිට නැවත නැවතත් element එක සෙවීමට අවශ්‍ය නොවන අතර, කෙලින්ම `button.addEventListener()` ලෙස ඊළඟ පියවර ලිවිය හැක.

12. Events

Concept Explanation (Sinhala): Event එකක් කියන්නේ web page එකේ සිදුවන යම්කිසි විශේෂ ක්‍රියාවකටයි.

Events සඳහා උදාහරණ:

- පරිශීලකයා button එකක් click කිරීම (click event).
- පරිශීලකයා input box එකක යමක් type කිරීම.
- Web page එක සම්පූර්ණයෙන්ම load වීම.
- Mouse pointer එක element එකක් උඩින් ගමන් කිරීම.
- Form එකක් submit කිරීම.

Sprint 1 වලදී අප අවධානය යොමු කරන්නේ ප්‍රධානම event එකක් වන `click` event එකටයි.

13. `addEventListener()`

Concept Explanation (Sinhala): `addEventListener()` මඟින් යම් element එකක් මත සිදුවන event එකක් සිදුවන තෙක් බලා සිටීමට (listen කිරීමට) JavaScript වලට ඉඩ සලසයි.

Code Block:

```
button.addEventListener("click", () => {
  console.log("Button clicked");
});
```

13.1 Parts of `addEventListener()`

Concept Explanation (Sinhala): ඉහත කේතයේ ප්‍රධාන කොටස් 3ක් ඇත:

1. `button` : අප සවන් දෙන (listen කරන) element එක.
2. `"click"` : අප බලාපොරොත්තුවෙන් සිටින event එක.
3. `() => { ... }` : click එක සිදු වූ විගස ක්‍රියාත්මක විය යුතු කේත කොටස (arrow function).

සරලවම: **"බොත්තම click කරන තුරු බලා සිටින්න, click කළ සැනින් curly braces `{ }` ඇතුළේ ඇති කේතය ක්‍රියාත්මක කරන්න."**

14. Arrow Function Used in Click Events

Concept Explanation (Sinhala): අප භාවිත කරන මෙම රටාව හඳුන්වන්නේ Arrow Function එකක් ලෙසයි.

Code Block:

```
() => {
  console.log("Button clicked");
}
```

Code Breakdown (Sinhala): Sprint 1 වලදී, මෙය **"පසුව ක්‍රියාත්මක වීමට නියමිත කුඩා කේත කොටසක් (block of code)"** ලෙස සලකන්න. එය ක්‍රියාත්මක වන්නේ අදාළ button එක click කළ විට පමණි.

15. `alert()`

Concept Explanation (Sinhala): `alert()` මඟින් browser එක උඩ කුඩා popup message එකක් (අනතුරු ඇඟවීමේ පණිවිඩයක්) දර්ශනය කරයි.

Code Block:

```
alert("Button clicked!");
```

15.1 Difference Between `console.log()` and `alert()`

Concept Explanation (Sinhala): මෙම ක්‍රම දෙකේ ප්‍රධාන වෙනස පහත පරිදි වේ:

Tool	Where the message appears	Main use
`console.log()`	Browser Console එක ඇතුළත	Developer ට පරීක්ෂා කිරීමට (Debugging)
`alert()`	පිටුව උඩ Popup එකක් ලෙස	User ට කෙලින්ම පණිවිඩයක් පෙන්වීමට

Code Block:

```
console.log("Button clicked");
alert("You clicked the button!");
```

Code Breakdown (Sinhala): මෙමඟින් එකවර ක්‍රියා දෙකක් සිදු වේ:

1. Console එකේ developer සඳහා සටහනක් වැටේ.
2. පිටුව මත පරිශීලකයාට popup එකක් දර්ශනය වේ.

16. Complete Sprint 1 Starter Example

Concept Explanation (Sinhala): මෙන්න Sprint 1 වල ඉගෙන ගත් දේවල් සියල්ල එකතු කර හදපු සම්පූර්ණ උදාහරණයක්. මේ සඳහා `index.html` නමින් file එකක් සාදා පහත කේතය ලියන්න.

Code Block:

```
<!DOCTYPE html>
<html>
<body>

  <!-- HTML goal: show a heading, paragraph, and button -->
  <h1>JavaScript Practice</h1>
  <p id="message">Click the button to run JavaScript.</p>
  <button id="runBtn">Run</button>

  <script>
    // JavaScript goal: react when the user clicks the button
    console.log("Page loaded");

    const button = document.getElementById("runBtn");
    console.log("Button ready");

    button.addEventListener("click", () => {
      console.log("Button clicked");
      alert("You clicked the button!");
    });
  </script>

</body>
</html>
```

17. Understanding the Complete Example

Concept Explanation (Sinhala): ඉහත කේතය ක්‍රියාත්මක වන ආකාරය කොටස් වශයෙන් තේරුම් ගනිමු.

17.1 HTML Heading

- `<h1>JavaScript Practice</h1>` : පිටුවේ ප්‍රධාන මාතෘකාව පෙන්වයි.

17.2 Paragraph

- `<p id="message">Click the button to run JavaScript.</p>` : සාමාන්‍ය ඡේදයක් පෙන්වන අතර, පසුව අවශ්‍ය වුවහොත් සොයා ගැනීමට `message` යනුවෙන් ID එකක් ලබා දී ඇත.

17.3 Button

- `<button id="runBtn">Run</button>` : "Run" යනුවෙන් ලියූ බොත්තමක් සාදන අතර, එහි ID එක `runBtn` වේ.

17.4 Page Loaded Log

- `console.log("Page loaded");` : පිටුව load වූ සැනින් මෙම පණිවිඩය console එකේ සටහන් වේ. එයින් JavaScript සාර්ථකව ක්‍රියාත්මක වන බව තහවුරු වේ.

17.5 Finding the Button

- `const button = document.getElementById("runBtn");` : JavaScript මඟින් HTML බොත්තම සොයාගෙන `button` නමැති variable එකේ රඳවා ගනියි.

17.6 Button Ready Log

- `console.log("Button ready");` : බොත්තම සාර්ථකව සොයාගත් බව සනාථ කිරීමට console එකේ සටහන් වේ.

17.7 Listening for the Click

- `button.addEventListener("click", () => { ... });` : බොත්තම click කරන තෙක් බලා සිටී. Click කළ සැනින්:
  1. Console එකේ `Button clicked` ලෙස වැටේ.
  2. Browser එකේ alert popup එකක් දර්ශනය වේ.

18. Expected Output of the Starter Example

Concept Explanation (Sinhala): පිටුව load වූ සැනින් console එකේ මේවා දිස්විය යුතුය:

```
Page loaded
Button ready
```

Button එක click කළ පසු console එකේ සහ පිටුවේ මේවා දිස්විය යුතුය:

- Console එකේ: `Button clicked`
- පිටුව මත Popup එකක් ලෙස: `You clicked the button!`

19. Common Beginner Error: Wrong ID

Concept Explanation (Sinhala): ආරම්භකයින් අතින් සිදුවන ප්‍රධානතම වැරැද්දක් වන්නේ ID එක නොගැලපීමයි (Wrong ID).

Code Block:

```
<!-- HTML -->
<button id="runBtn">Run</button>

<!-- JavaScript (Broken Code) -->
<script>
  const button = document.getElementById("wrongId");
</script>
```

Code Breakdown (Sinhala): මෙහිදී HTML හි ID එක `runBtn` වුවත්, JavaScript හි සොයන්නේ `wrongId` නමින් බැවින් element එක සොයා ගැනීමට නොහැකි වේ.

19.1 What Happens When JavaScript Finds Nothing?

Concept Explanation (Sinhala): `getElementById()` මඟින් අදාළ element එක සොයාගත නොහැකි වූ විට, එය විසින් `null` යන අගය ලබා දෙයි. `null` යන්නෙහි සරල අදහස **"කිසිවක් නැත" (Nothing was found)** යන්නයි.

එවිට, `const button = document.getElementById("wrongId");` යන්නෙහි ප්‍රතිඵලය `const button = null;` බවට පත්වේ.

ඉන්පසු මෙම කේතය ක්‍රියාත්මක වීමට උත්සාහ කළ විට: `button.addEventListener("click", ...)`

එය හරියට `null.addEventListener("click", ...)` ලෙස ක්‍රියා කිරීමට තැත් කිරීමකි. නමුත් `null` යනු බොත්තමක් නොවන බැවින්, එයට click event listen කළ නොහැක. එවිට browser console එකේ Error එකක් දර්ශනය වේ.

19.2 How to Fix the Wrong ID Error

Concept Explanation (Sinhala): මෙම වැරැද්ද නිවැරදි කිරීමට, HTML හි ID එක සහ JavaScript හි ලියන ID එක අකුරෙන් අකුර සමාන බව සහතික කර ගන්න.

- **නිවැරදි ක්‍රමය:**

  ```
  const button = document.getElementById("runBtn");
  ```

පහත කරුණු කෙරෙහි විශේෂයෙන් සැලකිලිමත් වන්න:

- අක්ෂර වින්‍යාසය (spelling)
- කැපිටල්/සිම්පල් අකුරු (capitalization - උදා: `runBtn` සහ `runbtn` එකිනෙකට වෙනස් වේ)
- අනවශ්‍ය හිස්තැන් (extra spaces)
- Quotes දැමීමට අමතක වීම (`""`)

20. Mini-Build: Start and Stop Buttons

Concept Explanation (Sinhala): මෙම කුඩා පාඩමෙන් බොත්තම් දෙකක් භාවිතයෙන් එකිනෙකට වෙනස් ක්‍රියාකාරකම් දෙකක් සිදු කරන ආකාරය ඉගෙන ගනිමු.

Code Block:

```
<!DOCTYPE html>
<html>
<body>

  <h1>Start / Stop Practice</h1>
  <button id="startBtn">Start</button>
  <button id="stopBtn">Stop</button>

  <script>
    console.log("Page loaded");

    const startButton = document.getElementById("startBtn");
    const stopButton = document.getElementById("stopBtn");

    startButton.addEventListener("click", () => {
      console.log("Start");
    });

    stopButton.addEventListener("click", () => {
      console.log("Stop");
    });
  </script>

</body>
</html>
```

20.1 What This Mini-Build Teaches

Concept Explanation (Sinhala): සෑම බොත්තමකටම (element එකකටම) තමන්ටම ආවේණික වූ event listener එකක් වෙන වෙනම තබා ගත හැකි බව මෙයින් පැහැදිලි වේ.

- **Start** button එක click කළ විට console එකේ `Start` ලෙස වැටේ.
- **Stop** button එක click කළ විට console එකේ `Stop` ලෙස වැටේ.

21. Changing Text on the Page with `textContent`

Concept Explanation (Sinhala): JavaScript වලට web page එකේ අපිට පෙනෙන්න තියෙන අකුරු (visible text) වෙනස් කරන්න පුළුවන් හැකියාවක් තියෙනවා. ඒ සඳහා භාවිත කරන්නේ `textContent` කියන property එකයි.

Code Block:

```
paragraph.textContent = "The button was clicked!";
```

Code Breakdown (Sinhala): මෙමඟින් `paragraph` කියන element එක ඇතුළේ තියෙන text එක "The button was clicked!" ලෙස ක්ෂණිකව වෙනස් වේ.

21.1 Text Change Example

Concept Explanation (Sinhala): බොත්තමක් click කළ විට පිටුවේ ඇති අකුරු වෙනස් වන ආකාරය මෙම උදාහරණයෙන් බලමු:

Code Block:

```
<!DOCTYPE html>
<html>
<body>

  <h1>Text Change Practice</h1>
  <p id="statusText">Waiting...</p>
  <button id="changeBtn">Change Text</button>

  <script>
    const paragraph = document.getElementById("statusText");
    const button = document.getElementById("changeBtn");

    button.addEventListener("click", () => {
      paragraph.textContent = "The button was clicked!";
      console.log("Paragraph text changed");
    });
  </script>

</body>
</html>
```

21.2 How the Text Change Example Works

Concept Explanation (Sinhala): 1. `statusText` ID එක මඟින් paragraph එක සොයා ගනියි. 2. `changeBtn` ID එක මඟින් button එක සොයා ගනියි. 3. Button එක click වන තුරු බලා සිටී. 4. Click කළ විට:

- `paragraph.textContent = ...` මඟින් පිටුවේ තිබූ "Waiting..." යන්න "The button was clicked!" ලෙස වෙනස් කරයි.
- `console.log(...)` මඟින් console එකට පණිවිඩයක් යවයි.

22. Inline Click Handler vs `addEventListener()`

Concept Explanation (Sinhala): Button click එකක් කෙලින්ම HTML tag එක ඇතුළේ ලියන්නත් පුළුවන් (Inline click handler):

Code Block:

```
<button onclick="alert('Button clicked!')">Click me</button>
```

නමුත් වඩාත් නිවැරදි සහ ප්‍රායෝගික ක්‍රමය වන්නේ JavaScript event listener එකක් ලියන එකයි:

```
<button id="runBtn">Click me</button>

<script>
  const button = document.getElementById("runBtn");
  button.addEventListener("click", () => {
    alert("Button clicked!");
  });
</script>
```

ඇයි `addEventListener()` වඩාත් සුදුසු වෙන්නේ? මක්නිසාද යත්, එයින් HTML (ව්‍යුහය) සහ JavaScript (ක්‍රියාකාරීත්වය) එකිනෙකින් වෙන් කර වෙන වෙනම පවත්වාගෙන යාමට (separation of concerns) උපකාරී වන බැවිනි.

23. Practice Goal for Sprint 1

Concept Explanation (Sinhala): Sprint 1 සාර්ථකව අවසන් කිරීමට නම් නිකම්ම කියවීම පමණක් ප්‍රමාණවත් නොවේ. පහත දේ ප්‍රායෝගිකව කරන්න:

1. කේත ඔබ විසින්ම type කරන්න.
2. ප්‍රතිඵලය ගැන අනාවැකි පළ කරන්න.
3. Browser එකේ පිටුව run කරන්න.
4. Console එක open කර බලාගන්න.
5. Buttons click කර වෙනස්කම් නිරීක්ෂණය කරන්න.
6. Errors තිබේ නම් ඒවා නිවැරදි කරන්න.

24. Warm-Up Drills

Concept Explanation (Sinhala): * **Drill 1: Create Basic HTML** (සරල HTML එකක් සැකසීම)

```
<!DOCTYPE html>
<html>
<body>
  <h1>My First JavaScript Page</h1>
  <p>This page will use JavaScript.</p>
  <button id="runBtn">Run</button>
</body>
</html>
```

- **Drill 2: Print Three Console Messages** (Console පණිවිඩ 3ක් දැමීම)

  ```
  console.log("Page loaded");
  console.log("Button ready");
  console.log("Button clicked");
  ```

- **Drill 3: Add Comments** (Comments එක් කිරීම)

  ```
  <!-- HTML goal: show a heading, paragraph, and button -->
  ```

  ```
  // JavaScript goal: react when the user clicks the button
  ```

25. Debug Challenge

Concept Explanation (Sinhala): **අභියෝගය:** `getElementById("runBtn")` යන්න `getElementById("wrongId")` ලෙස වෙනස් කර, `button.addEventListener` ක්‍රියා විරහිත වීමට හේතුව පැහැදිලි කරන්න.

පිළිතුර: HTML හි `runBtn` තිබුණත් JavaScript හි සොයන්නේ `wrongId` බැවින්, කිසිවක් හමු නොවේ. එවිට JavaScript විසින් `null` අගය ලබා දෙන අතර, `null` වලට `addEventListener` පාවිච්චි කළ නොහැකි බැවින් කේතය බිඳ වැටේ (fails).

26. Mini-Build Task

Concept Explanation (Sinhala): වැඩපිළිවෙළ: බොත්තම් දෙකක් සාදන්න. එකක් click කළ විට console එකේ `Start` ද, අනෙකෙන් `Stop` ද සටහන් විය යුතුය.

Code Block:

```
<!DOCTYPE html>
<html>
<body>

  <h1>Start and Stop</h1>
  <button id="startBtn">Start</button>
  <button id="stopBtn">Stop</button>

  <script>
    const startButton = document.getElementById("startBtn");
    const stopButton = document.getElementById("stopBtn");

    startButton.addEventListener("click", () => {
      console.log("Start");
    });

    stopButton.addEventListener("click", () => {
      console.log("Stop");
    });
  </script>

</body>
</html>
```

27. Extra Challenge

Concept Explanation (Sinhala): **අභියෝගය:** බොත්තමක් click කළ විට වෙනස් වන paragraph text එකක් සාදන්න.

Code Block:

```
<!DOCTYPE html>
<html>
<body>

  <h1>Change Text</h1>
  <p id="message">Original text</p>
  <button id="changeBtn">Change Text</button>

  <script>
    const message = document.getElementById("message");
    const button = document.getElementById("changeBtn");

    button.addEventListener("click", () => {
      message.textContent = "The text has changed!";
      console.log("Text changed");
    });
  </script>

</body>
</html>
```

28. Full Sprint 1 Practice Project

Concept Explanation (Sinhala): මෙම ව්‍යාපෘතිය මඟින් Sprint 1 හි ඉගෙන ගත් සියලුම සිද්ධාන්ත එකට එකතු කර ප්‍රායෝගිකව පෙන්වයි.

Code Block:

```
<!DOCTYPE html>
<html>
<body>

  <!-- HTML goal: create a small interactive page -->
  <h1>Sprint 1 Practice</h1>
  <p id="statusText">No button clicked yet.</p>

  <button id="helloBtn">Hello</button>
  <button id="learnBtn">Learning JavaScript</button>
  <button id="byeBtn">Goodbye</button>

  <script>
    // JavaScript goal: listen for button clicks and update the page
    console.log("Page loaded");

    const statusText = document.getElementById("statusText");
    const helloButton = document.getElementById("helloBtn");
    const learnButton = document.getElementById("learnBtn");
    const byeButton = document.getElementById("byeBtn");

    console.log("Buttons ready");

    helloButton.addEventListener("click", () => {
      console.log("Hello");
      statusText.textContent = "Hello button was clicked.";
    });

    learnButton.addEventListener("click", () => {
      console.log("Learning JavaScript");
      statusText.textContent = "You are learning JavaScript.";
    });

    byeButton.addEventListener("click", () => {
      console.log("Goodbye");
      statusText.textContent = "Goodbye button was clicked.";
    });
  </script>

</body>
</html>
```

29. Common Sprint 1 Mistakes

Concept Explanation (Sinhala): * **වැරැද්ද 1: වැරදි ID භාවිතය (Wrong ID)** HTML හි `runBtn` තිබියදී JavaScript හි `runButton` ලෙස සෙවීම. අකුරු හරියටම සමාන විය යුතුය.

- **වැරැද්ද 2: ID එක සෙවීමේදී Quotes අමතක වීම** `document.getElementById(runBtn);` (වැරදියි) -> `document.getElementById("runBtn");` (නිවැරදියි)
- **වැරැද්ද 3:** `console.log()` **පිටුව මත දිස්වේ යැයි සිතීම** `console.log()` පෙන්වන්නේ browser console එකේ පමණි. පිටුවේ text වෙනස් කිරීමට `textContent` පාවිච්චි කරන්න.
- **වැරැද්ද 4: Console එක විවෘත කිරීමට අමතක වීම** `console.log()` වල output බැලීමට හැමවිටම Inspect -> Console යා යුතුය.
- **වැරැද්ද 5: Element එකක් සෑදීමට පෙර එය සෙවීම (script පිහිටීම)** HTML element එක සෑදීමට පෙර JavaScript ක්‍රියාත්මක වුවහොත් element එක සොයාගත නොහැක. ඒ නිසා හැමවිටම `<script>` tag එක `<body>` එකේ අවසානයට ලිවිය යුතුය.

30. Key Vocabulary

Concept Explanation (Sinhala): * **HTML** : Web page එකේ structure එක හදන භාෂාව.

- **CSS** : Web page එක style කරන්න (ලස්සන කරන්න) පාවිච්චි කරන භාෂාව.
- **JavaScript** : Web page එකට behavior (ක්‍රියාකාරීත්වය) එකතු කරන භාෂාව.
- **Console** : Developers ලාට messages සහ errors බලාගන්න තියෙන තැන.
- `console.log()` : Console එකට පණිවිඩයක් යවන JavaScript ක්‍රමයක්.
- **Comment** : Code එක ඇතුළේ ලියන, browser එකෙන් ක්‍රියාත්මක නොකරන සටහන්.
- **Semicolon** (`;`) : Statement එකක අවසානය සලකුණු කරන ලකුණ.
- **Statement** : JavaScript වල තනි උපදෙසක් (instruction).
- **id** : HTML element එකකට දෙන අනන්‍ය නම.
- **document** : Web page එක නියෝජනය කරන JavaScript object එක.
- `getElementById()` : ID එකක් මඟින් element එකක් සොයා ගන්නා ක්‍රමය.
- **Event** : පිටුවේ සිදුවන යම් ක්‍රියාවක් (උදා: click කිරීම).
- **Click Event** : පරිශීලකයා element එකක් click කළ විට සිදුවන event එක.
- `addEventListener()` : Event එකක් සිදුවන තෙක් බලා සිට ක්‍රියාවක් කිරීමට ඇති ක්‍රමය.
- **Arrow Function** (`() => {}`) : කෙටි ක්‍රමයකට ලියන function එකක්.
- `alert()` : Browser එකේ popup පණිවිඩයක් පෙන්වන ක්‍රමයක්.
- `textContent` : HTML element එකක ඇති අකුරු කියවීමට හෝ වෙනස් කිරීමට ඇති ක්‍රමයක්.
- `null` : "කිසිවක් නැත" (nothing) යන අදහස දෙන අගය.

31. Sprint 1 Mental Model

Concept Explanation (Sinhala): සිතීමේ රාමුව (Mental Model) සරලව මෙසේය:

1. HTML මඟින් button එක හදයි.
2. JavaScript මඟින් button එක සොයාගනියි.
3. JavaScript මඟින් button එක click කරන තුරු බලා සිටී (listens).
4. පරිශීලකයා button එක click කරයි.
5. JavaScript මඟින් අදාළ කේතය ක්‍රියාත්මක කරයි.
6. Web page එක ප්‍රතිචාර දක්වයි.

32. Sprint 1 Revision Questions

Concept Explanation (Sinhala): ස්වයං ඇගයීම සඳහා පහත ප්‍රශ්න වලට තනිවම පිළිතුරු දීමට උත්සාහ කරන්න:

1. HTML වලින් කරන්නේ කුමක්ද?
2. CSS වලින් කරන්නේ කුමක්ද?
3. JavaScript වලින් කරන්නේ කුමක්ද?
4. HTML සහ CSS වලට තනියම කළ නොහැකි, JavaScript මඟින් පිටුවට එකතු කරන දේ කුමක්ද?
5. `console.log()` මඟින් කරන්නේ කුමක්ද?
6. `console.log()` වල output එක බලාගත හැක්කේ කොහෙන්ද?
7. Comments වැදගත් වන්නේ ඇයි?
8. JavaScript වල single-line comment එකක් ලියන සලකුණ කුමක්ද?
9. Semicolon එකකින් සිදුවන්නේ කුමක්ද?
10. HTML `id` එකක් යනු කුමක්ද?
11. IDs අනන්‍ය (unique) විය යුත්තේ ඇයි?
12. `document.getElementById("runBtn")` මඟින් සිදුවන්නේ කුමක්ද?
13. JavaScript වැරදි ID එකක් සෙවුවහොත් කුමක් සිදුවේද?
14. මෙම පාඩමට අනුව `null` යන්නෙන් අදහස් කරන්නේ කුමක්ද?
15. Event එකක් යනු කුමක්ද?
16. Click event එකක් යනු කුමක්ද?
17. `addEventListener("click", ...)` මඟින් සිදුවන්නේ කුමක්ද?
18. `alert()` මඟින් කරන්නේ කුමක්ද?
19. `alert()` සහ `console.log()` අතර වෙනස කුමක්ද?
20. `textContent` මඟින් කරන්නේ කුමක්ද?

33. Sprint 1 Self-Check Answers

Concept Explanation (Sinhala): ඉහත ප්‍රශ්න සඳහා කෙටි පිළිතුරු පහත පරිදි වේ:

1. HTML මඟින් පිටුවේ structure එක සහ content එක සාදයි.
2. CSS මඟින් පිටුවේ style එක (පෙනුම) පාලනය කරයි.
3. JavaScript මඟින් පිටුවට behavior සහ interactivity එකතු කරයි.
4. බොත්තම් click කිරීම් වැනි ක්‍රියාවලට ප්‍රතිචාර දැක්වීමේ හැකියාව එකතු කරයි.
5. Browser console එකේ අගයක් හෝ පණිවිඩයක් print කරයි.
6. Comments මඟින් කේතය කියවන මිනිසුන්ට එය පැහැදිලි කරයි (browser එක එය මඟහරියි).
7. Structure යනු HTML ය, Style යනු CSS ය, Behavior යනු JavaScript ය.
8. ID එකක් ආධාරයෙන් HTML element එකක් සොයාගනියි.
9. Element එක හමු නොවන අතර, JavaScript විසින් `null` ලබා දෙයි.
10. Click එකක් සිදුවන තෙක් බලා සිට, එය සිදු වූ විට අදාළ කේතය run කරයි.

34. Final Sprint 1 Checklist

Concept Explanation (Sinhala): ඔබට ඊළඟ Sprint එකට යාමට පෙර පහත දෑ තනිවම කළ හැකිදැයි පරීක්ෂා කරන්න:

- [ ] HTML, CSS, සහ JavaScript අතර වෙනස පැහැදිලි කිරීම.
- [ ] සරල HTML පිටුවක් සෑදීම.
- [ ] Heading, paragraph, සහ button එකක් එකතු කිරීම.
- [ ] `<script>` tag එකක් ඇතුළේ JavaScript ලිවීම.
- [ ] `console.log()` පාවිච්චි කිරීම සහ console එක open කර බැලීම.
- [ ] Comments සහ Semicolons නිවැරදිව ලිවීම.
- [ ] HTML element එකකට ID එකක් දී එය `document.getElementById()` හරහා සෙවීම.
- [ ] `addEventListener()` මඟින් click event එකක් සෑදීම.
- [ ] `alert()` පාවිච්චි කිරීම.
- [ ] `textContent` මඟින් පිටුවේ text වෙනස් කිරීම.
- [ ] වැරදි ID එකක් සෙවූ විට code එක fail වන හේතුව පැහැදිලි කිරීම.

35. One-Sentence Summary

Concept Explanation (Sinhala): Sprint 1 සාරාංශය: HTML මඟින් web page එක සාදන අතර, CSS මඟින් එය ලස්සන කරන අතර, JavaScript මඟින් බොත්තම් click කිරීම් වැනි පරිශීලකයාගේ ක්‍රියාවලට පිටුව ප්‍රතිචාර දක්වන තත්ත්වයට පත් කරයි.